EECS/BioE/MechE 106A/206A

UR-aMAZE-ing: Time-Optimal Ball-in-Maze Navigation on the UR7e with Collision-Aware MPC

Fall 2026

1 Team Information

Name	Email	Background
TienChun(leo) Chen A	leotcchen@berkeley.edu	Leo is an EECS M.Eng. student whose primary focus is on reinforcement learning for humanoid robots and simulation pipelines.
Jing Xu	jing_xu1112@berkeley.edu	Jing is an EECS M.Eng student with a background in Computer Science and AI. She has experience in robotics software development and reinforcement learning, and is particularly interested in intelligent agents, planning, and autonomous systems.
Chelsea Yan	chelseayan@berkeley.edu	Chelsea is a Mechanical Engineering M.Eng. student with a focus on robotics and control systems. She has experience in Python programming, ROS 2, robot kinematics, and state estimation, with a strong interest in motion planning and model-based control for robotic systems.
Zhuoren Li	gabriel.li@berkeley.edu	Zhuoren is a ME M.Eng student focusing in Robotics and motion planning. He has experience in motion planning and control for humanoid robots.
Steven Wu	chiachien_wu@berkeley.edu	Steven is a ME M.Eng student who is interested in robotics and control theory. He's now working on the control of self-driving cars for his capstone project.

2 Abstract

We will make a UR7e solve a ball-in-maze puzzle it has never seen, as fast as possible, without dropping the ball into a hole. The Robotiq gripper holds a 50 × 50 cm maze board by its side edge. An overhead webcam maps the maze and tracks the ball, an A* planner finds a safe route through convex safe corridors, and a collision-aware Model Predictive Contouring Controller (MPCC) tilts the board to minimize completion time while predicting and avoiding wall collisions and holes. Because the board is gripped at its edge, it must rotate about a remote pivot, which requires coordinated motion of all six joints. The full pipeline is first validated in a MuJoCo simulation that uses the same ROS 2 interface as the robot. Compared with last semester's **Hold UR Pla7e**, which balanced a ball on a flat plate along a human-drawn path with unconstrained controllers, we move from balancing to autonomous, time-optimal navigation under obstacle constraints.

3 Project Description

What are your project goals?

We will make a UR7e solve a ball-in-maze puzzle it has never seen, as fast as possible, without dropping the ball into a hole; completion time is always reported together with completion rate. *Must-have:* tilt a side-gripped board on command, track the ball, map the maze, plan a path with A*, and reach the goal with PID and then a constrained MPC. *Research:* a time-optimal controller (MPCC), a ball-wall impact model, and replanning after disturbances.

Stretch: moving the board horizontally as a second input.

How will you design your project in terms of software/hardware architecture (at a high level)?

The Robotiq gripper holds a light 50 × 50 cm maze board by a 3D-printed handle on its edge. An overhead Logitech C922 (60 fps) and four AR tags on the board corners observe the board and ball. ROS 2 packages handle perception, estimation, planning, control and arm motion, in a one-time setup stage per maze and a real-time loop at the camera rate (Figure 1). During development, a MuJoCo simulation replaces the robot, board and ball behind the same ROS 2 interface.

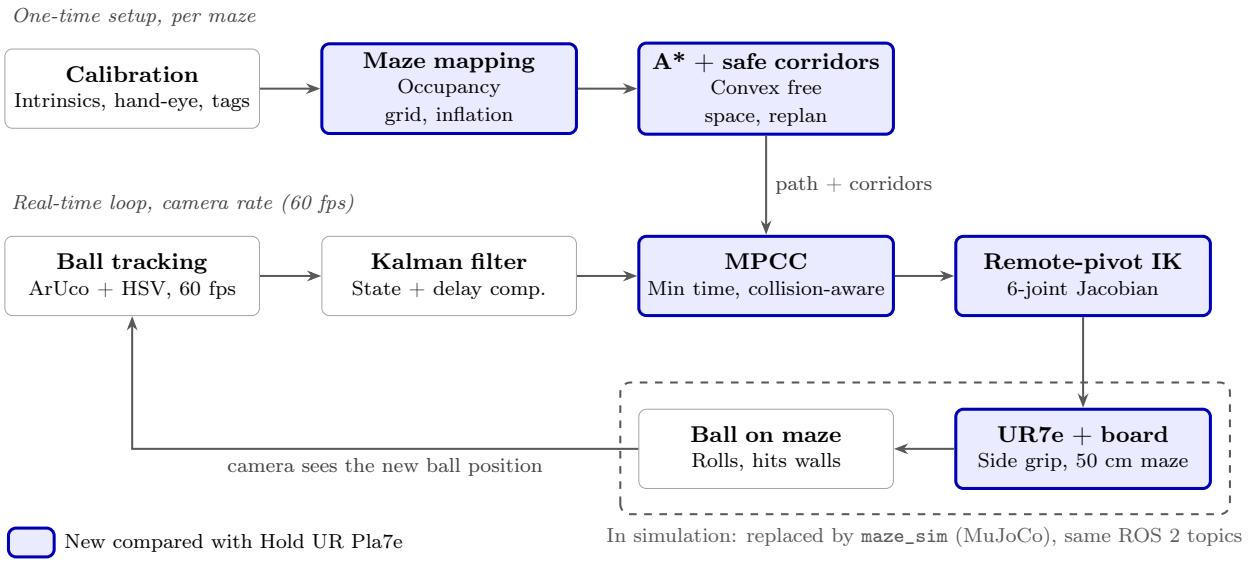


Figure 1: System architecture: one-time setup stage and real-time loop. Highlighted boxes are new compared with Hold UR Pla7e; tracking and the Kalman filter build on its approach. The dashed region is replaced by the MuJoCo simulator during offline validation.

How does your project incorporate sensing, planning, and actuation?

Sensing: the camera turns the maze into a grid of walls, holes and free space, and tracks the ball by color in board coordinates every frame (no RealSense is needed, since the ball stays on the board plane); a Kalman filter estimates velocity and compensates for delay.
Planning: A* finds a path away from holes, wrapped in rectangular safe corridors, and replans if the ball leaves them.
Actuation: because the board is held at its edge, all six joints move together to tilt it about its own center.
Control: we build three MPC controllers, each extending the previous one:
(1) Constrained tracking MPC: which keeps the predicted ball inside the corridors;
(2) MPCC: which rewards progress so the ball speeds up on straights and slows for turns; and
(3) Collision-aware MPCC: which adds a model of wall bounces.

How will you test or assess your project? What constitutes a success? What are some realistic goals? What are some reach goals?

We compare four controllers (PID, LQR, constrained MPC and MPCC) on a single maze whose only hole is the goal. Each controller runs 50 times in MuJoCo and then 10–20 times on the robot. All runs start from the same position and are logged, so differences in the results come from the controller alone. A run succeeds if the ball drops into the goal hole within a set time limit. For each controller we report completion rate, mean completion time over successful runs, and unplanned wall impacts per run. This shows how much the wall constraints (MPC compared with PID and LQR) and the time-optimal objective (MPCC compared with MPC) each contribute.

Realistic goals: PID, LQR and the constrained MPC all running on the robot and compared on the maze; the

ball reaching the goal hole in at least 80% of runs with the constrained MPC; and the constrained MPC achieving a higher completion rate and fewer wall impacts than PID and LQR.
Reach goals: MPCC lowering the mean completion time by at least 20% compared with MPC at the same completion rate, and fewer wall impacts with the impact-aware prediction.

4 Tasks

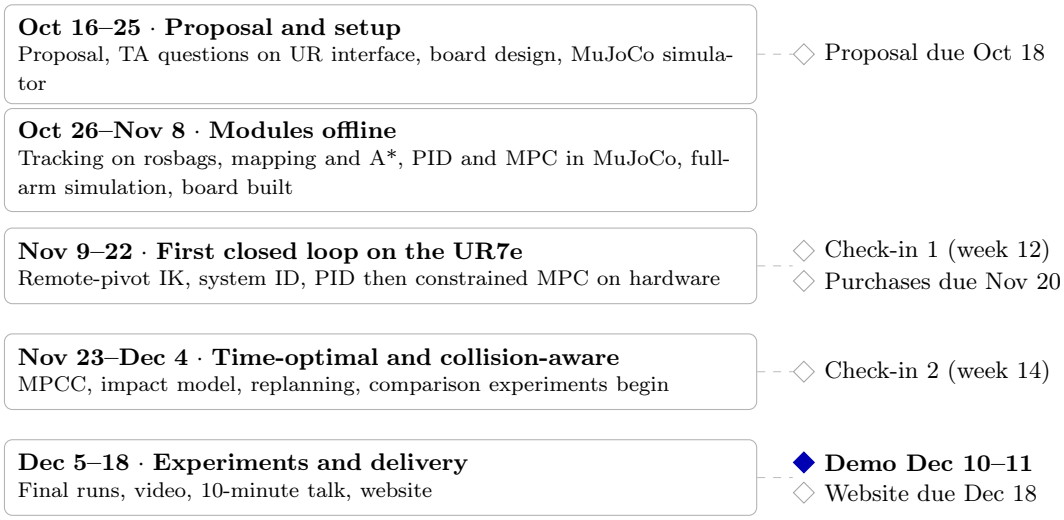


Figure 2: Project timeline: five phases and six course gates.

1. **Proposal and setup.**
   (a) **Submit proposal.** Finalize and submit the project proposal.
   (b) **Staff questions.** Ask TAs whether the UR7e accepts high-rate streamed joint commands or only trajectories; confirm hardware approval requirements with ESG.
   (c) **Design the board and handle.** Define the board layout, form-closure gripper handle, and weight budget.
   (d) **Build the simulator.** Implement MuJoCo board-only mode (board, walls, holes, ball) with the `maze_sim` ROS 2 interface.
2. **Develop modules offline.**
   (a) **Perception.** Develop ball tracking and occupancy-grid maze mapping using recorded images; check geometry on degraded MuJoCo images against ground truth.
   (b) **Planning.** Implement A* path planning, path smoothing, and safe corridor generation.
   (c) **Control in simulation.** Implement a PID baseline and initial constrained tracking MuJoCo board-only mode.
   (d) **Full simulation.** Add the UR arm to MuJoCo; validate remote-pivot IK and the PID closed loop through the complete simulation pipeline; verify the command interface using URSim or mock hardware.
   (e) **Build the board.** Laser-cut the base, 3D-print the handle and walls, and apply the felt surface.
3. **First closed loop on the UR7e.**
   (a) **Remote-pivot IK.** Implement forward kinematics, determine a safe home pose, and develop resolved-rate control with safety limits.
   (b) **System identification.** Fit a first-order lag and delay model from sine and step tilt commands; incorporate the identified model into MuJoCo.
   (c) **PID on hardware.** Demonstrate ball tracking and PID-based closed-loop navigation on a simple, repeatable maze using the UR7e.

(d) **Constrained MPC on hardware.** Test Layer 1 with corridor constraints if the PID hardware baseline is reliable.

4. **Time-optimal and collision-aware control.**
   (a) **MPCC.** Implement Layer 2, tune it in MuJoCo, and validate it on physical hardware.
   (b) **Impact model and replanning.** Implement Layer 3 and recovery from disturbances; measure ball-wall restitution on the real board.
   (c) **Begin comparison experiments.** Demonstrate the full pipeline from an unseen maze image to MPCC navigation, and collect initial controller comparison data.

5. **Experiments and delivery.**
   (a) **Final runs.** Compare controllers across all test mazes in MuJoCo and on hardware, recording completion rates, times, and tracking performance.
   (b) **Demo.** Present the working system and deliver the 10-minute presentation.
   (c) **Website.** Publish the project report, demonstration videos, and experimental results.

5 Bill of Materials

5.1 Use of Lab Resources

Item	Quantity	Notes
UR7e (w/ Robotiq parallel gripper)	1	Booked sessions; gripper and arm unmodified
Logitech C922 webcam + tripod	1	Overhead camera, 720p at 60 fps
AR tags	4	Printed from bCourses files

5.2 Other Purchases

All items stay within the $55 team budget; prices are estimates to confirm before the 11/20 purchase deadline. The gripper handle and wall pieces are 3D printed at Supernode (covered when reported to ESG, $0).

Item	Quantity	Justification
3 mm plywood or acrylic sheet	1	50 × 50 cm maze base, laser cut at CSW (about $15–25)
Felt sheet	1	Board surface that adds rolling resistance (about $8, Amazon)
Steel balls or marbles, 12–16 mm	1 set	Contrasting color for tracking (about $10, Amazon or McMaster)
Spare maze materials	–	Second maze layout, from remaining budget

6 Other

6.1 Motivation and Prior Work

A ball in a maze is a compact model of real manipulation: an underactuated object, moved only through contact, among obstacles, with delayed vision. Solving it quickly forces sensing, planning and control to work together. Building on prior work, our project integrates autonomous maze perception and planning with constrained, progress-maximizing MPC on a general-purpose robotic arm.

Project	Robot / Actuation	Perception / Planning	Control	Gap Addressed
Hold UR Pla7e (EECS 206A)	UR7e; plate fixed to wrist; wrist joints only	ArUco + HSV tracking; human-defined path	PID; per-axis linear MPC; residual RL	No obstacles, planning, or time optimization

Project	Robot / Actuation	Perception / Planning	Control	Gap Addressed
Gaber et al. (ETH Zurich)	Dedicated two-motor labyrinth	Known maze; color tracking; Kalman filter	Two-level nonlinear MPC; 28% success on hard maze	No wall-impact model; tilt-only platform
CyberRunner (ETH)	Dedicated two-motor labyrinth	Camera; learned dynamics model	Model-based RL	Learning-based rather than classical control
Sawyer's Travels (Northwestern)	Sawyer arm; maze on gripper; joints 4–5	Webcam; RRT on 7×7 grid	Two PID loops	No MPC or time optimization

Our contributions:

1. **From balancing to autonomous navigation.** The robot sees an unseen maze, builds an occupancy grid, plans a route and replans when the ball drifts.
2. **Time-optimal objective.** MPCC maximizes progress along the path instead of tracking a fixed-speed reference.
3. **Collision-aware MPC.** Walls and holes are constraints inside a coupled 2D MPC; as a research extension we model ball-wall impacts, the ETH paper's main open problem.
4. **Side-grip kinematics.** The board rotates about a remote pivot about 30 cm from the gripper, requiring coordinated motion of all six joints.
5. **Full 6-DoF actuation (stretch).** Board translation adds an inertial force on the ball, which a tilt-only labyrinth cannot produce; this is why an arm is the right robot for the task.

6.2 Risks and Fallbacks

The two largest risks are the moment of a 50 cm board on the gripper and the arm's command latency; both are tested in the first robot session.

Risk	Effect	Fallback
Heavy or oversized board	Slip or flex	Use a lighter 30–40 cm board and a secure handle.
High command latency	Slower response	Model the delay and reduce tilt and speed limits.
Slow or unstable MPCC	Poor control	Fall back to constrained tracking MPC.
Inaccurate impact model	Poor rebound prediction	Report the limitations and retain Layer 2.
Ball tracking loss	Uncertain state	Improve lighting and contrast; predict briefly, then stop the trial if tracking remains lost.
Sim-to-real mismatch	Reduced performance	Recalibrate on hardware, increase safety margins, and report results separately.
Limited robot access	Less tuning time	Develop with MuJoCo and rosbags; prioritize hardware calibration and validation.

6.3 Software and Reference Resources

Software: ROS 2, UR ROS 2 driver, MuJoCo, OpenCV, NumPy/SciPy, OSQP, CasADi/acados, and RViz.

We will build on existing open-source infrastructure while focusing our contributions on autonomous planning, constrained predictive control, and remote-pivot kinematics. Reused components will be documented and distinguished from our original work.

Resource	**Reference / Reusable Components**	**Application**
Hold UR Pla7e `real/vision/`	ArUco homography, HSV ball detection, tilt-aware Kalman filter	Ball tracking and state estimation
Hold UR Pla7e `ball_balance_controller`	Jacobian control, ROS trajectory streaming, UR driver interface	Remote-pivot control and robot commands
Hold UR Pla7e `mock_ball_publisher.py` and RViz configuration	Offline testing, visualization, hardware configuration	Offline validation and `maze_bringup`
Hold UR Pla7e `ballplate/controllers/mpc`	Unconstrained MPC formulation	Baseline controller
MuJoCo and MuJoCo Menagerie	Contact simulation, rolling friction, camera rendering, UR5e model	`maze_sim`; model adaptation and UR7e validation
Gaber, Bi, D'Andrea (ETH)	Obstacle-constrained MPC, Kalman filtering	Safety constraints and benchmarking
Liniger et al. (MPCC)	MPCC formulation and racing controller	Progress-maximizing objective
OSQP MPC example	Sparse quadratic programming, warm starting	Constrained MPC and MPCC solvers
Sawyer's Travels	Vision-based maze detection, grid planning	Maze mapping and path planning

References

[1] R. Colletti, A. Remmerie, X. Ma, and A. Gasca Rosas, "Hold UR Pla7e: Ball-on-Plate Control with a UR7e," EECS 206A Final Project, University of California, Berkeley, 2026. [Online]. Available: https://github.com/collettiriccardo/hold-ur-pla7e

[2] J. Gaber, T. Bi, and R. D'Andrea, "Adaptive nonlinear model predictive control for a real-world labyrinth game," in *Proc. 63rd IEEE Conf. Decision and Control (CDC)*, 2024, pp. 7478–7483, doi: 10.1109/CDC56724.2024.10886880.

[3] T. Bi and R. D'Andrea, "Sample-efficient learning to solve a real-world labyrinth game using data-augmented model-based reinforcement learning," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2024, pp. 7455–7460, doi: 10.1109/ICRA57147.2024.10610577.

[4] A. Liniger, A. Domahidi, and M. Morari, "Optimization-based autonomous racing of 1:43 scale RC cars," *Optimal Control Applications and Methods*, vol. 36, no. 5, pp. 628–647, 2015, doi: 10.1002/oca.2123.

[5] K. Öfjäll and M. Felsberg, "Combining vision, machine learning and automatic control to play the labyrinth game," arXiv:1604.00975, 2016. [Online]. Available: https://arxiv.org/abs/1604.00975

[6] E. Chien, M. Dai, A. Pervan, W. Wang, and M. Wiznitzer, "Sawyer's Travels," Northwestern University, ME 495 Course Project, 2017.

[7] B. Stellato, G. Banjac, P. Goulart, A. Bemporad, and S. Boyd, "OSQP: An operator splitting solver for quadratic programs," *Mathematical Programming Computation*, vol. 12, no. 4, pp. 637–672, 2020, doi: 10.1007/s12532-020-00179-2.

[8] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033, doi: 10.1109/IROS.2012.6386109.